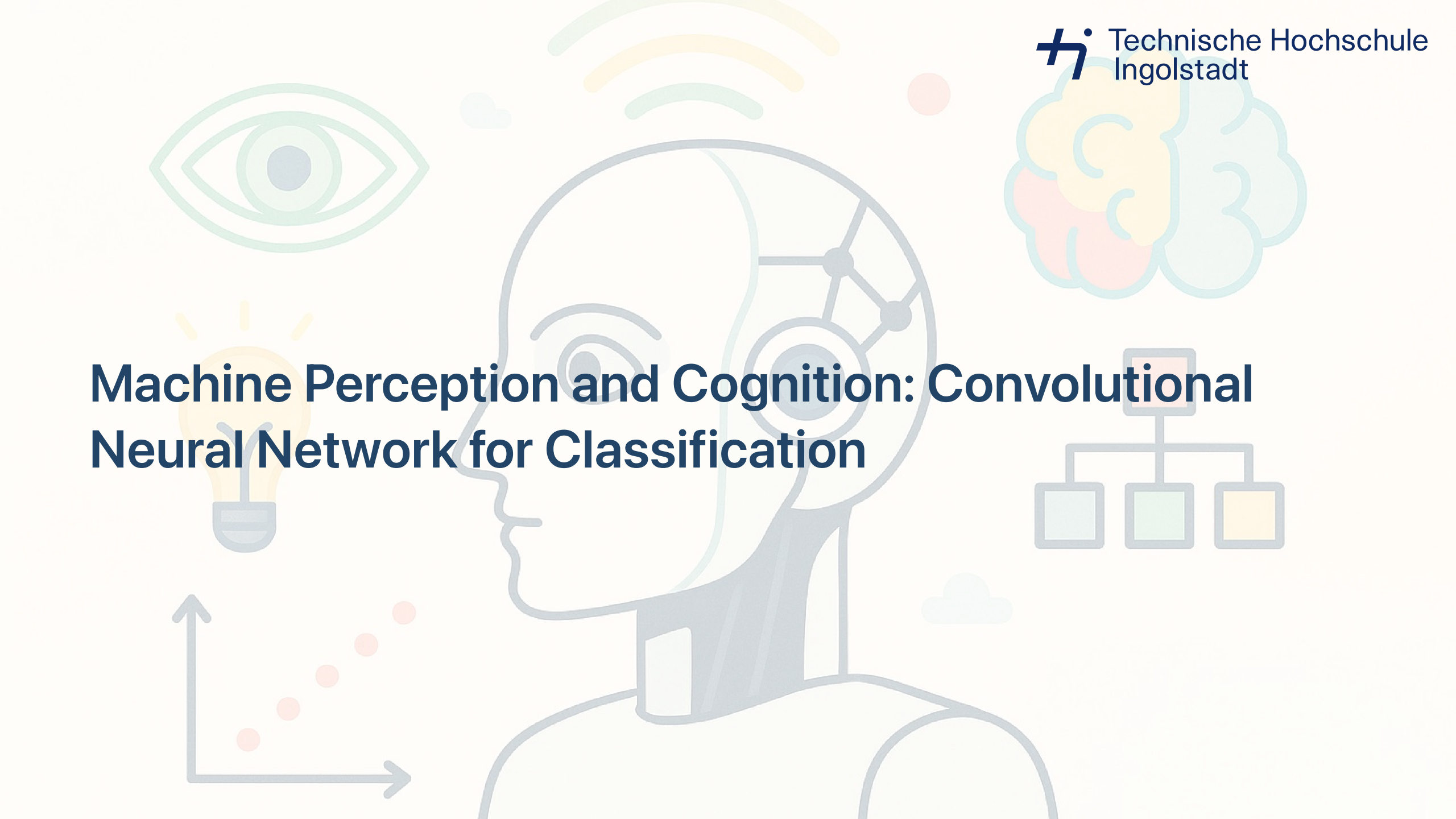
Technische Hochschule Ingolstadt



Machine Perception and Cognition: Convolutional Neural Network for Classification

Learning objectives

After this lecture you will be able to:

- 🎯 Explain the **multi-class classification** problem and its differences from binary classification
- 🏗️ Design a **CNN architecture** for image classification with K classes
- 🔢 Derive the **softmax + cross-entropy** loss used in multi-class classification
- 🛠️ Implement a full **training pipeline** in PyTorch: data loading, augmentation, training loop, and evaluation
- 📊 Evaluate model performance using **accuracy, confusion matrix, and F1 score**
- 🔄 Apply **transfer learning** to adapt pretrained models to new datasets

Multi-class classification

Definition: given an input $\mathbf{x}$ (e.g., an image), predict its class from a fixed set of K mutually exclusive categories.

Examples:

- Recognize a handwritten digit (0–9): $K = 10$ classes
- Classify objects in images (car, pedestrian, bicycle, ...): K can be 10, 100, or 1000
- Identify a species from a photo: K can be thousands

Formally: learn a mapping $f : \mathcal{X} \to \{1, \ldots, K\}$ from inputs to class labels.

In practice, the network outputs a **probability vector** $\hat{\mathbf{y}} \in \mathbb{R}^K$ with $\sum_k \hat{y}_k = 1$ and $\hat{y}_k \geq 0$.

From binary to multi-class

Binary classification ($K = 2$) outputs a single probability via the sigmoid function:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

Multi-class classification ($K > 2$) outputs K probabilities via the **softmax** function:

$$\hat{y}_k = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \quad k = 1, \ldots, K$$

The K output scores $\mathbf{z} = (z_1, \ldots, z_K)^\top$ are called **logits**. The softmax maps them to a valid probability distribution: $\hat{y}_k > 0$ and $\sum_k \hat{y}_k = 1$.

The predicted class is: $\hat{c} = \arg\max_k \hat{y}_k$

Example dataset: CIFAR-10

A standard benchmark for multi-class image classification:

Property	Value
Classes (K)	10 (airplane, car, bird, cat, deer, dog, frog, horse, ship, truck)
Image size	32×32 pixels, RGB (3 channels)
Training set	50 000 images (5 000 per class)
Test set	10 000 images (1 000 per class)
Input dimension	$3 \times 32 \times 32 = 3072$ values

CIFAR-10 is **balanced**: every class has exactly the same number of samples.

The full CNN classification pipeline

$$\underbrace{\text{Raw image}}_{\mathbf{x} \in \mathbb{R}^{C \times H \times W}} \xrightarrow{\text{preprocess}} \underbrace{\text{Normalized tensor}}_{\mathbf{x}_{\text{norm}}} \xrightarrow{\text{CNN}} \underbrace{\text{Feature map}}_{\mathbf{h}} \xrightarrow{\text{head}} \underbrace{\text{Logits}}_{\mathbf{z} \in \mathbb{R}^{K}} \xrightarrow{\text{softmax}} \underbrace{\hat{\mathbf{y}}}_{\text{probs}}$$

The CNN consists of two parts:

1. **Feature extractor** (backbone): stacked convolutional layers learn hierarchical visual features (edges → textures → parts → objects)
2. **Classifier head**: one or more fully-connected layers map the feature representation to K class scores (logits)

The predicted label is $\hat{c} = \arg\max_k \hat{y}_k$ (no need to compute softmax at inference time).

Cross-entropy loss for multi-class classification

For K classes, the softmax turns logits $\mathbf{z} \in \mathbb{R}^K$ into class probabilities $\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$.

The **cross-entropy loss** between the true label distribution $\mathbf{y}$ and the predicted probabilities $\hat{\mathbf{y}}$ is:

$$\mathcal{L} = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$

For **one-hot encoded** labels, only the entry for the true class c is non-zero ($y_c = 1$, all others $y_k = 0$), so the sum collapses to:

$$\mathcal{L} = -\log \hat{y}_c = -\log \frac{e^{z_c}}{\sum_{j=1}^{K} e^{z_j}}$$

Intuition: the loss penalizes low probability assigned to the correct class.

Numerical stability: log-sum-exp

Computing $\hat{y}_k = e^{z_k} / \sum_j e^{z_j}$ directly can cause **numerical overflow** when logits are large.

Stable alternative — log-sum-exp trick: subtract the maximum logit before exponentiation:

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k - z_{\max}}}{\sum_{j=1}^{K} e^{z_j - z_{\max}}}$$

PyTorch's `nn.CrossEntropyLoss` combines logits → softmax → log → cross-entropy in a **single numerically stable operation** (log-softmax + NLL loss).

➡️ Always pass **raw logits** (not softmax outputs) to `nn.CrossEntropyLoss`.

CNN architecture for image classification

A typical architecture for CIFAR-10:

$$\underbrace{\text{Conv} \to \text{BN} \to \text{ReLU} \to \text{Pool}}_{\text{block 1}} \to \underbrace{\text{Conv} \to \text{BN} \to \text{ReLU} \to \text{Pool}}_{\text{block 2}} \to \underbrace{\text{Conv} \to \text{BN} \to \text{ReLU}}_{\text{block 3}} \to \underbrace{\text{GAP} \to \text{FC}}_{\text{head}}$$

CNN architecture for image classification

Key design decisions:

Choice	Typical value
Kernel size	3×3 or 5×5 (universal for vision)
Number of filters	32, 64, 128 (doubling after each pool)
Pooling	2×2 max-pooling or stride-2 conv
Normalization	Batch normalization after each conv
Classifier	Global average pooling + one FC layer
Output	K logits, no activation

Convolutional layers as feature extractors

Convolutional layers learn **hierarchical feature detectors**:

- **Early layers** (e.g. $l = 1, 2$): Detect local, low-level features
- **Middle layers** (e.g. $l = 3, 4$): Compose low-level features into parts
- **Deep layers** (e.g. $l \geq 5$): Detect semantic concepts

Each convolutional layer applies F filters, producing F feature maps. Stacking layers and pooling gradually **reduces spatial size** while **increasing the number of channels** — compressing spatial information into semantic content.

Global average pooling

Instead of flattening all feature map values, **global average pooling (GAP)** averages each feature map to a single value:

$$\mathrm{GAP}(\mathbf{A}^{(l)}) = \frac{1}{H \cdot W} \sum_{h=1}^{H} \sum_{w=1}^{W} \mathbf{A}^{(l)}_{:,h,w} \in \mathbb{R}^{F}$$

Advantages over flattening:

- Produces a **fixed-size** output regardless of input resolution
- Drastically **fewer parameters** in the classifier head (no large FC layers)
- Acts as **structural regularization**: each feature map must be globally meaningful
- The final FC layer becomes a simple **linear classifier** over the learned features

Batch normalization in CNNs

Batch normalization (BN) is applied **after each convolutional layer**, before the activation:

$$\hat{\mathbf{z}}^{(l)} = \frac{\mathbf{z}^{(l)} - \boldsymbol{\mu}_B}{\sqrt{\boldsymbol{\sigma}_B^2 + \epsilon}} \cdot \boldsymbol{\gamma} + \boldsymbol{\beta}$$

where $\boldsymbol{\mu}_B, \boldsymbol{\sigma}_B^2$ are computed over the batch for each channel, and $\boldsymbol{\gamma}, \boldsymbol{\beta}$ are learned scale/shift parameters.

Benefits:

- Reduces **internal covariate shift**
- Makes training significantly **more stable and faster**

In PyTorch: `nn.BatchNorm2d(num_features)`

Building a simple CNN in PyTorch

```
import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2),                                  # 32x32 → 16x16
            nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2),                                  # 16x16 → 8x8
            nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
        )
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),   # Global average pooling → 128x1x1
            nn.Flatten(),              # → 128
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        return self.head(self.features(x))
```

Data loading with torchvision

PyTorch's `torchvision` library provides built-in datasets, transforms, and data loaders:

```
import torchvision
import torchvision.transforms as T
from torch.utils.data import DataLoader

# Pixel values normalized to zero mean and unit variance
transform = T.Compose([
    T.ToTensor(),                              # [0,255] uint8 → [0,1] float
    T.Normalize(mean=[0.491, 0.482, 0.447],
                std=[0.247, 0.244, 0.262]) # CIFAR-10 channel statistics
])

train_set = torchvision.datasets.CIFAR10(root='./data', train=True,
                                         download=True, transform=transform)
test_set  = torchvision.datasets.CIFAR10(root='./data', train=False,
                                         download=True, transform=transform)

train_loader = DataLoader(train_set, batch_size=128, shuffle=True,  num_workers=4)
test_loader  = DataLoader(test_set,  batch_size=256, shuffle=False, num_workers=4)
```

Data augmentation

Training on a fixed dataset risks **overfitting**: the network memorizes training samples rather than learning generalizable features.

Data augmentation applies random transformations at training time, exposing the network to diverse variations of each image.

Data augmentation

Augmentation	Effect
Random horizontal flip	Invariance to left–right orientation
Color jitter (brightness, contrast, saturation)	Robustness to lighting changes
Random rotation	Orientation robustness
Gaussian blur	Robustness to focus/blur
Cutout / Random erasing	Robustness to occlusion

⚠️ Apply augmentations **only to the training set** — the validation and test sets use fixed preprocessing.

Data augmentation in PyTorch

```
train_transform = T.Compose([
    T.RandomHorizontalFlip(),
    T.RandomCrop(32, padding=4),        # pad by 4 pixels, then crop back to 32x32
    T.ColorJitter(brightness=0.2,
                  contrast=0.2,
                  saturation=0.2),
    T.ToTensor(),
    T.Normalize(mean=[0.491, 0.482, 0.447],
                std=[0.247, 0.244, 0.262])
])

val_transform = T.Compose([            # No random augmentations!
    T.ToTensor(),
    T.Normalize(mean=[0.491, 0.482, 0.447],
                std=[0.247, 0.244, 0.262])
])

train_set = torchvision.datasets.CIFAR10(..., transform=train_transform)
val_set   = torchvision.datasets.CIFAR10(..., transform=val_transform)
```

The training loop

```
import torch
import torch.nn as nn

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = SimpleCNN(num_classes=10).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(50):
    model.train() # switch dropout and batch-normalization to training behavior
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        logits = model(images)           # forward pass
        loss = loss_fn(logits, labels)   # cross-entropy loss
        optimizer.zero_grad()            # reset gradients
        loss.backward()                  # backpropagation
        optimizer.step()                 # weight update
```

Monitoring training — validation loop

After each epoch, evaluate on the **validation set** (separate from training and test sets):

```
def evaluate(model, loader, loss_fn, device):
    model.eval()   # disables dropout; uses running BN statistics
    total_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():   # no gradient computation needed
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            logits = model(images)
            total_loss += loss_fn(logits, labels).item() * len(labels)
            correct += (logits.argmax(dim=1) == labels).sum().item()
            total += len(labels)
    return total_loss / total, correct / total  # avg loss, accuracy
```

Key differences between training and evaluation algorithms

Key behavior differences between `model.train()` and `model.eval()`:

- **BatchNorm**: training uses batch statistics; eval uses tracked running mean/variance
- **Dropout**: training randomly drops units; eval passes all activations through

Monitoring training — loss and accuracy curves

Tracking **training loss**, **validation loss**, and **validation accuracy** over epochs reveals the training dynamics:

Pattern	Interpretation
Both losses decrease steadily	✅ Healthy training
Train loss ↓ but val loss ↑	⚠️ Overfitting — use regularization
Both losses plateau early	⚠️ Learning rate too small or model too simple
Loss oscillates or diverges	⚠️ Learning rate too large

Best practice: save the model checkpoint at the **lowest validation loss** (early stopping), not at the final epoch.

Learning rate scheduling

A constant learning rate is rarely optimal. **Learning rate schedules** reduce η over training:

Schedule	Description
Step decay	Multiply η by factor $\gamma < 1$ every n epochs
Cosine annealing	$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t / T))$
Warm-up + cosine	Linear warm-up for first k steps, then cosine decay
ReduceLROnPlateau	Reduce η when validation loss stops improving

Learning rate scheduling

```
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=50, eta_min=1e-5
)
# After optimizer.step() in each epoch:
scheduler.step()
```

Evaluating on the test set

The **test set** is used **only once** — at the very end — to estimate real-world performance.

Never use the test set during training or hyperparameter tuning. If you tune on the test set, the reported accuracy is optimistically biased.

Standard data split:

Split	Purpose	Typical fraction
Training set	Weight optimization	70–80 %
Validation set	Hyperparameter tuning, early stopping	10–15 %
Test set	Final unbiased evaluation	10–15 %

For small datasets ($N < 10^4$), use **k-fold cross-validation** to reduce estimation variance.

Accuracy

Accuracy is the fraction of correctly classified samples:

$$\text{Accuracy} = \frac{\text{number of correct predictions}}{\text{total number of samples}} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\left[\hat{c}^{(i)} = c^{(i)}\right]$$

- Easy to interpret; the default metric for balanced datasets
- **Misleading for imbalanced datasets**: a classifier that always predicts the majority class achieves high accuracy without learning anything

Example: if 95 % of images are "dog" and 5 % are "cat", a trivial model predicting "dog" always achieves 95 % accuracy but fails completely on "cat".

For imbalanced data: report **per-class accuracy**, **precision**, **recall**, and **F1 score**.

Confusion matrix

The **confusion matrix** $\mathbf{C} \in \mathbb{N}^{K \times K}$ shows how predictions relate to true labels:

$$C_{ij} = \text{number of samples with true class } i \text{ predicted as class } j$$

- Diagonal entries: **correct predictions**
- Off-diagonal entries: **misclassifications** (which class is confused with which)

Confusion matrix

Example (3 classes):

$$\mathbf{C} = \begin{pmatrix} 90 & 5 & 5 \\ 3 & 85 & 12 \\ 2 & 8 & 90 \end{pmatrix}$$

Row 2 (true class 2) has 12 samples misclassified as class 3 → systematic confusion between class 2 and 3.

The confusion matrix reveals **which classes are hard** and guides targeted improvements.

Precision, recall, and F1 score

For each class k, define:

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}, \qquad \text{Recall}_k = \frac{TP_k}{TP_k + FN_k}$$

$$F1_k = \frac{2 \cdot \text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}$$

Precision, recall and F1 score

Metric	Answers
Precision	"Of all predicted class k, how many are truly class k?"
Recall	"Of all true class k, how many did we detect?"
F1	Harmonic mean of precision and recall

Macro-averaged F1 averages $F1_k$ over all classes equally.

Weighted F1 weights each class by its frequency — useful for imbalanced datasets.

Dealing with class imbalance

Real-world datasets are often **imbalanced** — some classes are much rarer than others.

Strategies:

Strategy	Mechanism
Oversampling	Duplicate or synthesize minority-class samples (e.g., SMOTE)
Undersampling	Randomly remove majority-class samples
Class-weighted loss	Assign higher loss weight to rare classes: $\mathcal{L} = -\sum_k w_k y_k \log \hat{y}_k$
Data augmentation	Generate extra training samples for rare classes

Transfer learning — motivation

Training a deep CNN from scratch requires:

- **Large datasets** (hundreds of thousands of images)
- **Significant compute** (hours to days on a GPU)

In practice, most datasets are small (10^3–10^4 samples). Training from scratch leads to **overfitting**.

Transfer learning

Transfer learning exploits the fact that features learned on large datasets (e.g., ImageNet with 1.2 million images and 1000 classes) are **generic** and transfer well to new tasks:

- Low-level features (edges, textures) are **universally useful**
- Mid-level features (shapes, parts) transfer across similar domains
- High-level features (semantic concepts) are more task-specific

➡️ Start from a **pretrained model** instead of random weights.

Pretrained models and ImageNet

ImageNet Large Scale Visual Recognition Challenge (ILSVRC):

- 1.2 million training images, 1000 classes
- The de-facto standard pretraining dataset for visual recognition

Pretrained models and ImageNet

Widely used pretrained CNN architectures:

Model	Parameters
ResNet-50	25 M
EfficientNet-B0	5.3 M
MobileNetV3-Large	5.5 M
ConvNeXt-Tiny	28 M

All are available in `torchvision.models` with pretrained weights.

Feature extraction vs. fine-tuning

Two transfer learning strategies:

Feature extraction: freeze the pretrained backbone; only train the new classifier head.

- Suitable when the target dataset is **small** or **similar** to the pretraining dataset
- Fast: only a few parameters are updated

Fine-tuning: keep the pretrained weights as initialization, then train **all layers** (usually with a low learning rate).

- Suitable when the target dataset is **larger** or **somewhat different**
- Better performance at the cost of more compute and risk of overfitting

Common practice: first train the head only for a few epochs (feature extraction), then unfreeze all layers and fine-tune at a lower learning rate.

Fine-tuning a pretrained model in PyTorch

```python
import torchvision.models as models

# Load ResNet-50 with pretrained ImageNet weights
model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)

# Freeze all backbone parameters
for param in model.parameters():
    param.requires_grad = False

# Replace the classifier head for CIFAR-10 (10 classes)
in_features = model.fc.in_features        # 2048 for ResNet-50
model.fc = nn.Linear(in_features, 10)     # only this layer has requires_grad=True

# Train head only (phase 1)
optimizer = torch.optim.AdamW(model.fc.parameters(), lr=1e-3)

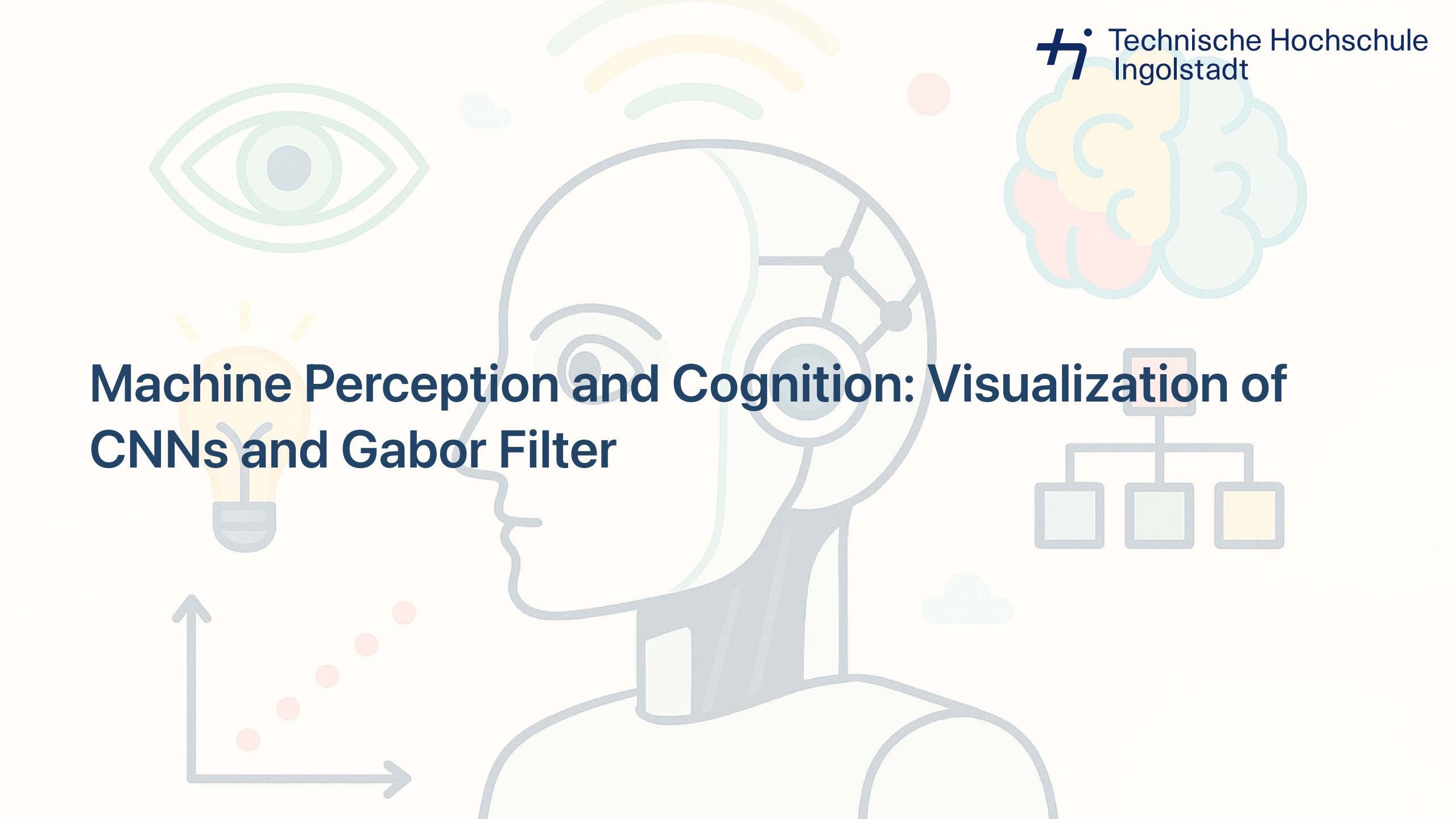
# Later: unfreeze all and fine-tune (phase 2)
for param in model.parameters():
    param.requires_grad = True
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
```

Convolutional Neural Network for Classification

Recap!

- 🎯 **Multi-class classification**: predict one of K classes; output a probability vector via softmax.
- 📉 **Cross-entropy loss**: $\mathcal{L} = -\log \hat{y}_c$; penalizes low probability on the correct class.
- 🏗️ **CNN architecture**: convolutional feature extractor + global average pooling + FC head.
- 📊 **Evaluation**: accuracy for balanced data; confusion matrix, precision, recall, and F1 for full picture.
- 🔄 **Transfer learning**: start from ImageNet pretrained weights; freeze backbone, train head, then fine-tune.

Technische Hochschule Ingolstadt



Machine Perception and Cognition: Visualization of CNNs and Gabor Filter

Why Visualize CNNs?

CNNs achieve remarkable performance — but what have they actually **learned**?

- A trained ResNet-50 can classify 1000 ImageNet categories with >75% top-1 accuracy
- Yet the weights are millions of floating-point numbers with no obvious interpretation

Motivations for visualization:

- 🔍 **Debugging**: identify failure modes and unexpected biases
- 🎓 **Scientific understanding**: what visual concepts does each layer encode?
- 🛡️ **Trust and safety**: verify that the model uses correct evidence for its decisions
- 🔄 **Improved design**: guide architecture choices based on what layers learn

The Black-Box Problem

A CNN maps an input image $\mathbf{x} \in \mathbb{R}^{C \times H \times W}$ to a score vector $\mathbf{z} \in \mathbb{R}^{K}$ through hundreds of non-linear transformations.

Problem: given a decision $\hat{c} = \arg\max_k z_k$, we cannot easily answer:

- Which pixels **caused** the prediction?
- Which **visual patterns** trigger a given neuron?
- Do early layers learn edges? Do later layers learn object parts?

Goal of CNN visualization: open the black box and make learned representations **human-interpretable**.

Taxonomy of CNN Visualization Methods

Category	Question answered	Example methods
Weight visualization	What does a filter look like?	Plot filter kernels directly
Activation visualization	What input maximizes a neuron?	Gradient ascent on input
Attribution / saliency	Which input pixels matter for a prediction?	Gradient-based saliency
Feature inversion	What image produces a given feature vector?	Deconvnet

Gabor Filters

A **Gabor filter** is a sinusoidal wave modulated by a Gaussian envelope:

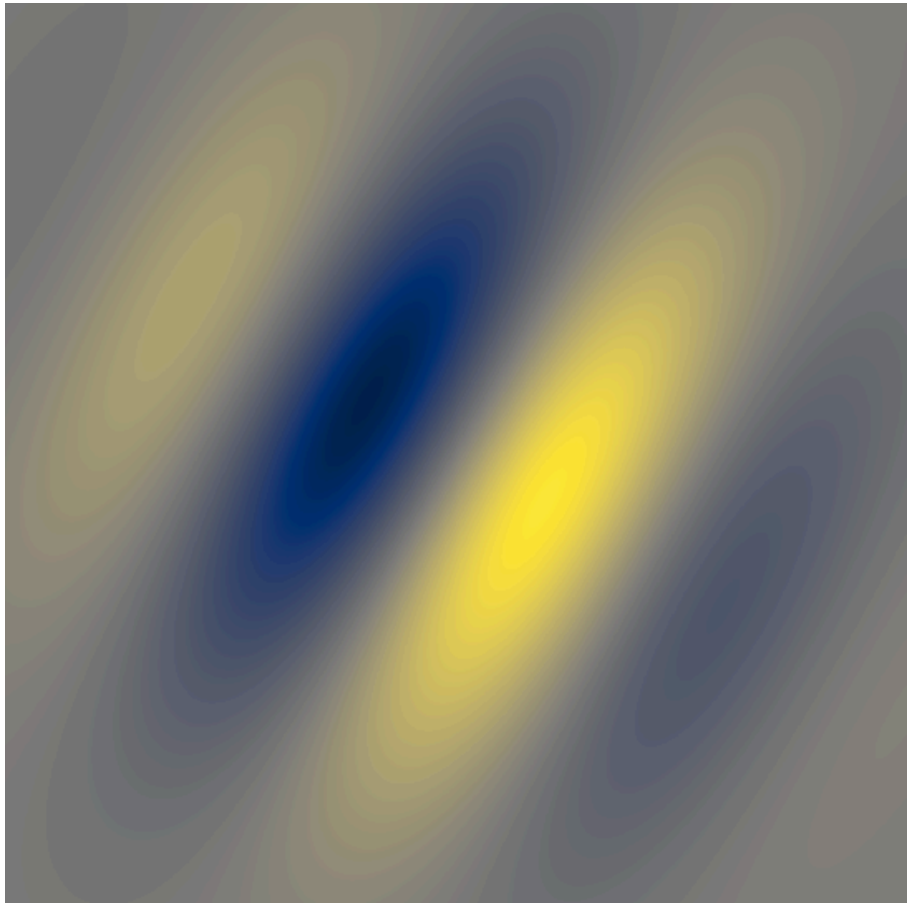


Image source: AkanoToE, CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0), via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Gabor_filter_function.png

Gabor Filters

A **Gabor filter** is a sinusoidal wave modulated by a Gaussian envelope:

$$g(x, y; \lambda, \theta, \psi, \sigma, \gamma) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \cos\left(\frac{2\pi x'}{\lambda} + \psi\right)$$

where $x' = x\cos\theta + y\sin\theta$ and $y' = -x\sin\theta + y\cos\theta$.

Key parameters:

- λ: wavelength of the sinusoid (spatial frequency)
- ψ: phase offset of the sinusoid
- θ: orientation of the filter
- σ: size of the Gaussian envelope (spatial extent)
- γ: spatial aspect ratio (ellipticity)



Gabor Filters

Gabor filters are **band-pass filters**: they respond to edges and textures at a specific frequency and orientation.

Image source: https://www.pexels.com/photo/white-and-black-digital-device-displaying-graph-70911/

Gabor Filters

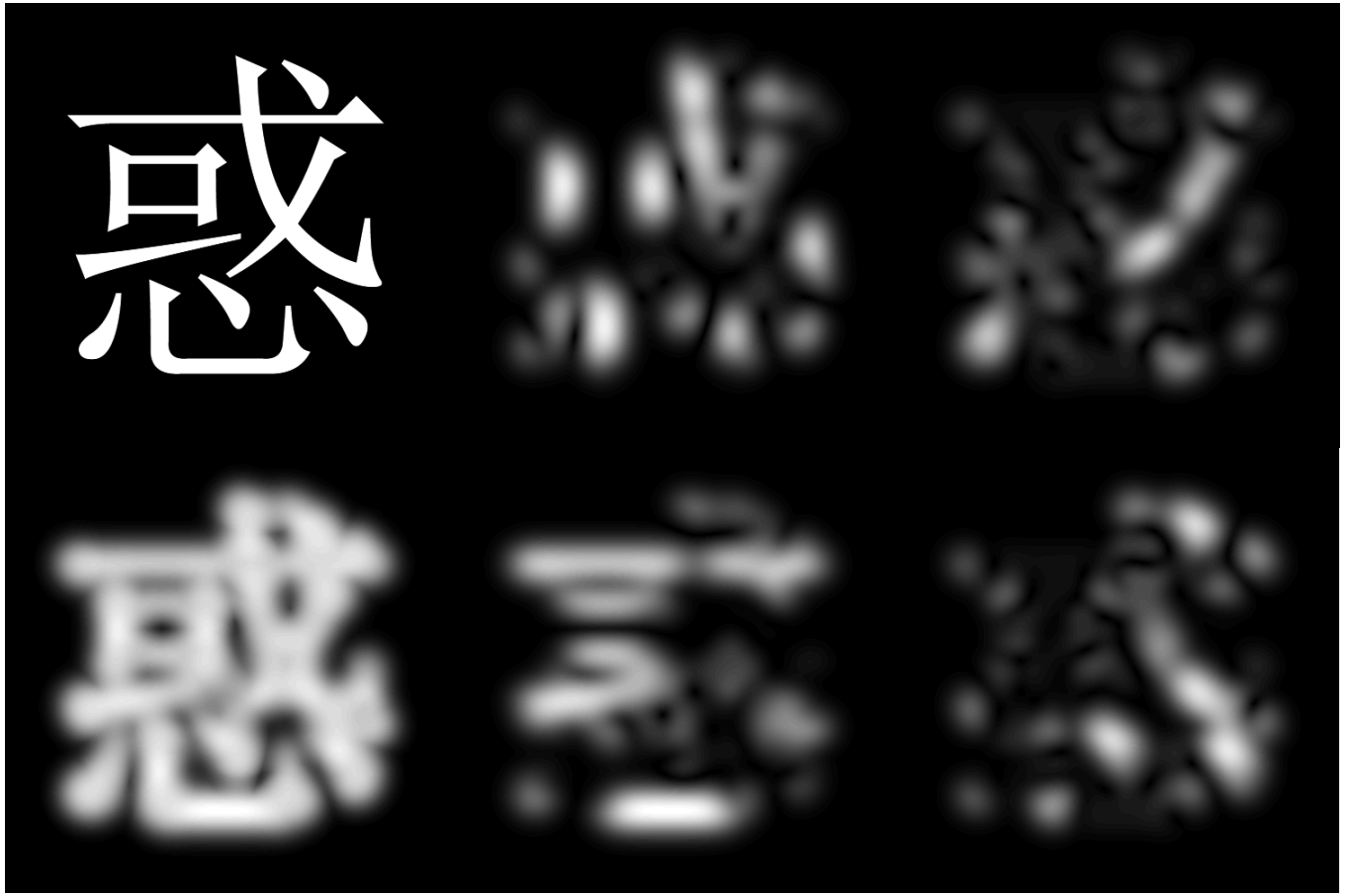


Image source: Johndoe154, Public domain, via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Gabor-ocr.png

Gabor Filters and Biological Vision

Gabor filters are not merely a signal-processing choice — they appear to match how biological vision systems work:

- It is believed that receptive fields in mammals correspond to Gabor-like functions — selectively responding to oriented edges at specific positions and frequencies
- Oriented, bandpass filters are thus thought to form the first processing stage of mammalian vision

➡️ This raises the question: do CNNs learn the same kind of filters?

Gabor Filters and Biological Vision

> Olshausen, B. A., & Field, D. J. (1996). Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature, 381*(6583), 607-609.

Online: https://www.nature.com/articles/381607a0.pdf

First-Layer Filters of a CNN

When we **directly visualize the learned weight tensors** $\mathbf{W}^{(1)} \in \mathbb{R}^{F \times C \times k \times k}$ of the first convolutional layer of a well-trained CNN:

- AlexNet (Krizhevsky et al., 2012): 96 filters of size $11 \times 11 \times 3$
- VGGNet, ResNet: 64 filters of size $3 \times 3 \times 3$

Empirical observation: first-layer filters strongly resemble Gabor filters — they encode:

- Oriented edges at multiple angles ($0^\circ, 45^\circ, 90^\circ, 135^\circ$)
- Color-opponent patterns (red-green, blue-yellow)
- Low-frequency luminance blobs

This convergence is not hand-designed; it emerges from gradient descent on natural images.

Why CNNs Learn Gabor-Like Filters

Sparse coding applied to natural image patches also yields Gabor-like basis functions (Olshausen & Field, 1996).

Explanation: natural images have strong statistical regularities:

- Nearby pixels are correlated (spatial structure)
- Oriented edges are the dominant low-level structure
- Energy is concentrated at low and mid spatial frequencies

A CNN trained on natural images must learn filters that are **statistically efficient** for representing these structures. Oriented bandpass filters (Gabor-like) are the optimal solution — the same solution believed to be found in biological visual systems.

➡️ The first convolutional layer of a CNN is effectively a **learned Gabor filter bank**.

Beyond the First Layer

Direct weight visualization works well only for the **first layer** (filters operate on raw pixels).

For deeper layers, filters operate on high-dimensional **feature maps**, not on interpretable pixel values. A filter in layer 5 cannot be directly plotted as an image.

Challenge: how do we visualize what deeper neurons detect?

Two seminal approaches:

1. **Gradient-based class model visualization and saliency maps** — Simonyan et al. (2014)
2. **Deconvolutional networks** — Zeiler & Fergus (2014)

Paper: Deep Inside Convolutional Networks

> Simonyan, K., Vedaldi, A., & Zisserman, A. (2013). Deep inside convolutional networks: Visualising image classification models and saliency maps. *arXiv preprint arXiv:1312.6034*.

Online: https://arxiv.org/pdf/1312.6034

Paper: Deep Inside Convolutional Networks

Two main contributions:

1. **Class model visualization**: synthesize an image that maximally activates a class score
2. **Image-specific saliency maps**: highlight which pixels in a given image are most important for its classification

Core idea: the gradient of the network output with respect to the input image encodes sensitivity information — it tells us how much each pixel influences the prediction.

Class Model Visualization

Goal: find an input image $\mathbf{x}^*$ that maximizes the score $S_c(\mathbf{x})$ for class c:

$$\mathbf{x}^* = \arg\max_{\mathbf{x}} \; S_c(\mathbf{x}) - \lambda \|\mathbf{x}\|_2^2$$

The ℓ_2 regularization term $\lambda \|\mathbf{x}\|_2^2$ prevents the optimization from producing extreme pixel values.

Algorithm (gradient ascent in input space):

1. Initialize $\mathbf{x}^{(0)}$ with random noise (or zeros)
2. Forward pass: compute $S_c(\mathbf{x}^{(t)})$
3. Backward pass: compute $\frac{\partial S_c}{\partial \mathbf{x}^{(t)}}$
4. Update: $\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \eta \left(\frac{\partial S_c}{\partial \mathbf{x}^{(t)}} - 2\lambda \mathbf{x}^{(t)} \right)$
5. Repeat until convergence

Class Model Visualization — Intuition

$$\mathbf{x}^* = \arg\max_{\mathbf{x}} \; S_c(\mathbf{x}) - \lambda \|\mathbf{x}\|_2^2$$

Key observations:

- The optimization is carried out **in pixel space**, not in weight space
- We keep the network **weights fixed** and optimize the **input**
- This is gradient ascent (not descent!) — we want to *increase* the class score
- The resulting images look unnatural to humans but reveal what the network "wants to see" for each class

Class Model Visualization — Examples

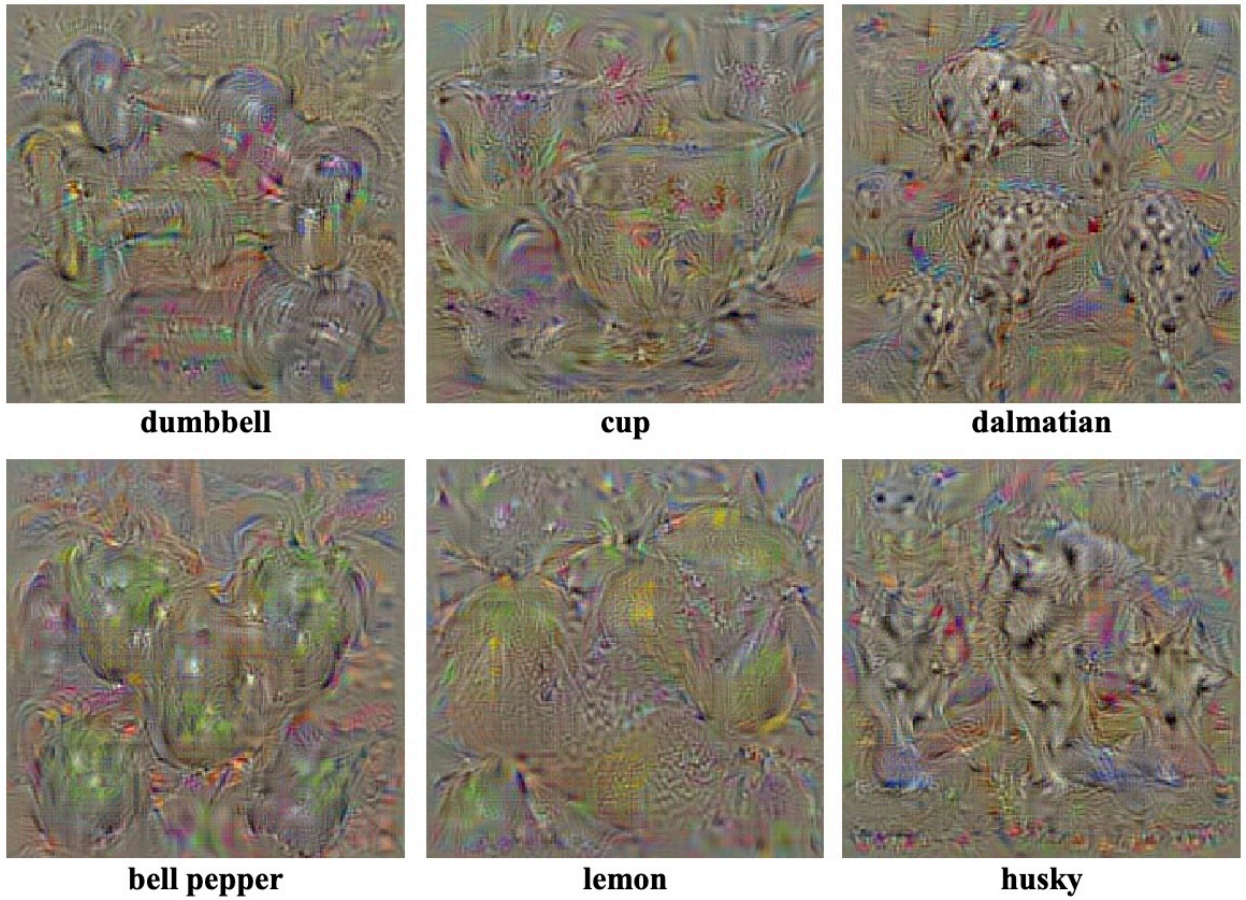


Image source: Simonyan, K., Vedaldi, A., & Zisserman, A. (2013). Deep inside convolutional networks: Visualising image classification models and saliency maps. arXiv preprint arXiv:1312.6034.

Class Model Visualization — Examples

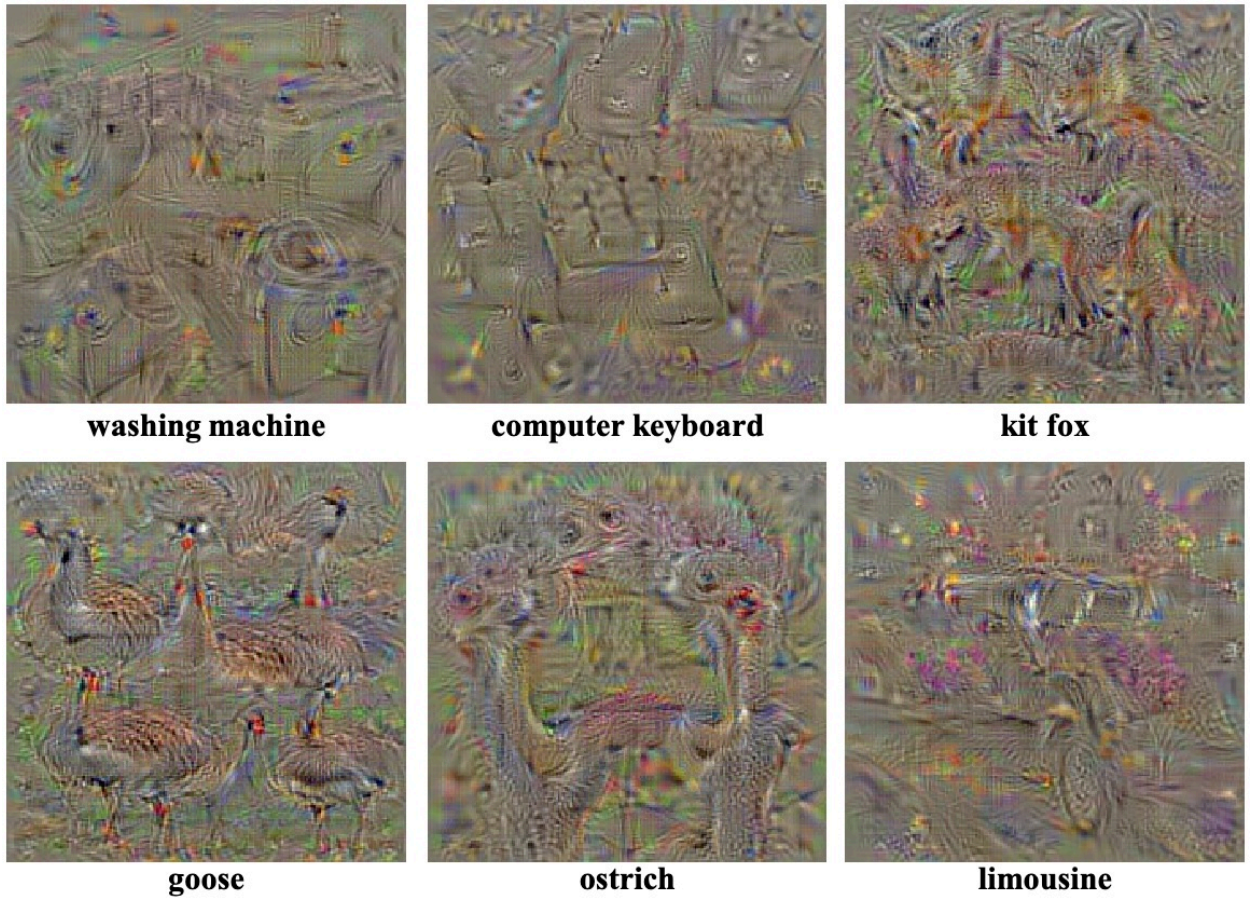


Image source: Simonyan, K., Vedaldi, A., & Zisserman, A. (2013). Deep inside convolutional networks: Visualising image classification models and saliency maps. arXiv preprint arXiv:1312.6034.

Image-Specific Saliency Maps

Goal: given a specific input image $\mathbf{x}_0$ and predicted class c, find which pixels contributed most to the score $S_c(\mathbf{x}_0)$.

Saliency map computation:

$$M_{ij} = \max_{ch} \left| \frac{\partial S_c}{\partial x_{ch,ij}} \right|$$

where the max is taken over color channels $ch \in \{1, \ldots, C\}$ to produce a single-channel map.

Saliency Maps — Algorithm

```python
import torch

def compute_saliency(model, image, class_idx):
    image = image.unsqueeze(0)  # add batch dimension
    image.requires_grad_(True)  # enable gradient w.r.t. input

    # Forward pass
    scores = model(image)       # shape: (1, K)
    score_c = scores[0, class_idx]

    # Backward pass: gradient of score w.r.t. input pixels
    model.zero_grad()
    score_c.backward()

    # Saliency: take max absolute gradient over color channels
    saliency = image.grad.data.abs()      # shape: (1, C, H, W)
    saliency, _max_indices = saliency.max(dim=1)  # shape: (1, H, W); index unused
    return saliency.squeeze()             # shape: (H, W)
```

Saliency Maps — Interpretation

The saliency map $M \in \mathbb{R}^{H \times W}$ assigns a **relevance score** to each pixel:

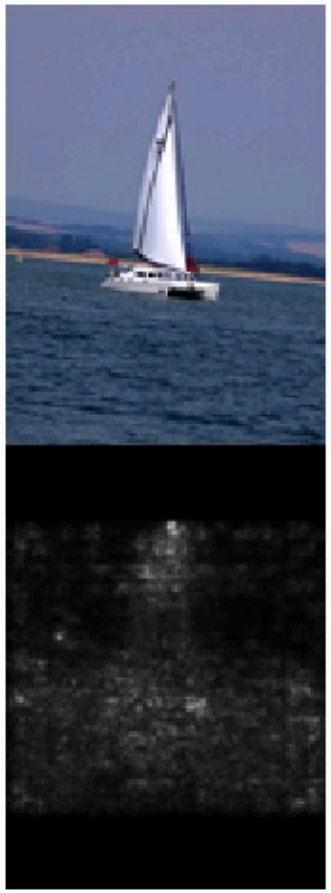
- **High saliency**: the pixel strongly influences the class score (small change → big effect on output)
- **Low saliency**: the pixel has little influence on the prediction

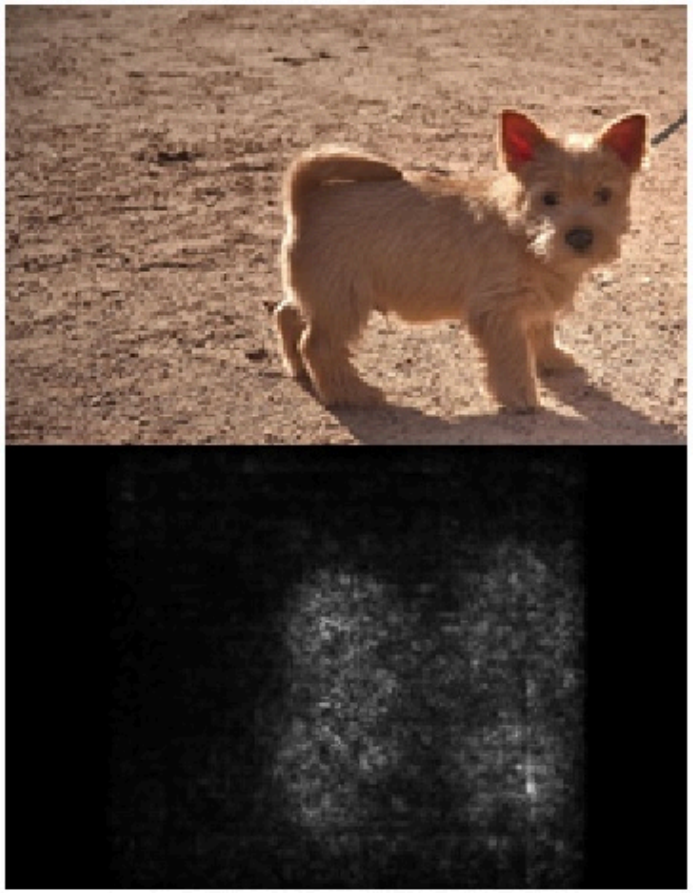
Saliency Maps — Interpretation

Limitations of vanilla gradient saliency:

- Gradients can be **noisy** — small perturbations cause large gradient fluctuations
- The map highlights pixels where the score is sensitive, not necessarily pixels that are *causally responsible*
- Saturation problem: gradients vanish for neurons with high activations (ReLU saturation)

Saliency Maps — Examples





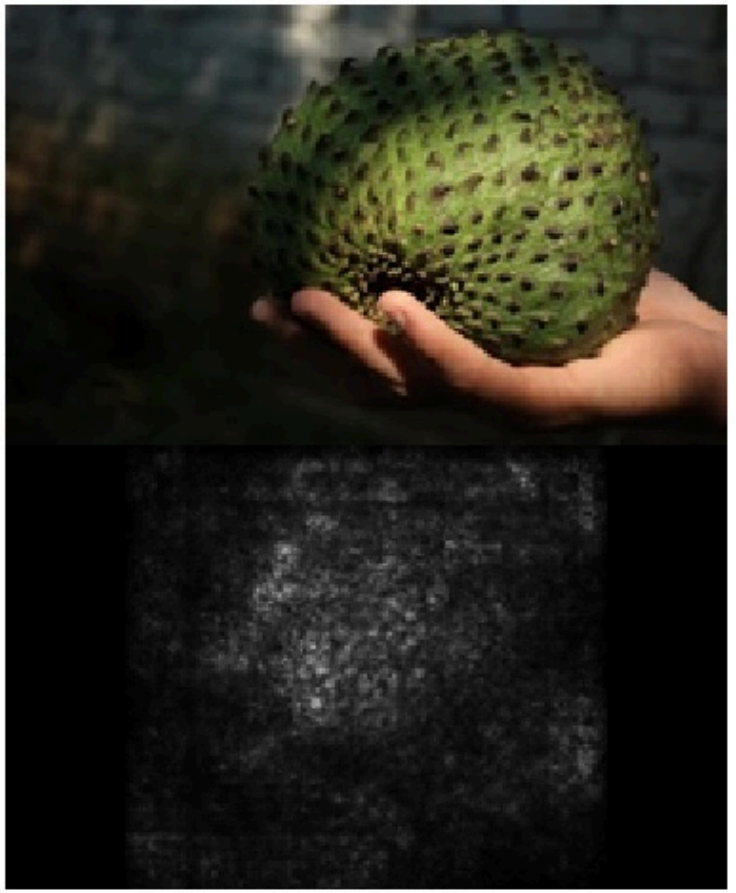


Image source: Simonyan, K., Vedaldi, A., & Zisserman, A. (2013). Deep inside convolutional networks: Visualising image classification models and saliency maps. arXiv preprint arXiv:1312.6034.

Paper: Visualizing and Understanding Convolutional Networks

> Zeiler, M. D., & Fergus, R. (2014, September). Visualizing and understanding convolutional networks. In *European conference on computer vision* (pp. 818-833). Cham: Springer International Publishing.

Online: https://www.matthewzeiler.com/mattzeiler/eccv2014.pdf

Paper: Visualizing and Understanding Convolutional Networks

Key contributions:

1. **Deconvolutional network (Deconvnet)**: a technique to map feature activations back to input pixel space
2. **Systematic visualization** of features learned at each layer of a CNN
3. **Empirical insights** about what CNN layers learn and how training progresses

The Deconvolutional Network

Idea: attach a **deconvnet** as a probe to each layer of a trained CNN.

To visualize which input patterns activate a given feature map:

1. **Forward pass**: compute all activations up to the target layer
2. **Select**: set all activations in the target feature map to zero except the one of interest
3. **Deconvnet pass**: propagate the selected activation back to pixel space using inverted operations

The Deconvolutional Network

The deconvnet contains three reverse operations corresponding to the forward network:

- **Unpooling** (inverse of max-pooling)
- **Rectification** (ReLU applied to the reconstruction signal)
- **Filtering** (inverse of convolution)

Deconvnet — Unpooling

Max-pooling (forward): $h_{ij} = \max_{(p,q)\in\mathcal{R}_{ij}} a_{pq}$

During the forward pass, the **location** of the maximum within each pooling region is recorded in **switch variables** s_{ij}.

Unpooling (backward approximation): place the value h_{ij} back at the location recorded by s_{ij}; all other positions in the unpooled map are set to zero.

$$\tilde{a}_{pq} = \begin{cases} h_{ij} & \text{if } (p,q) = s_{ij} \\ 0 & \text{otherwise} \end{cases}$$

This preserves the spatial structure of the max response and avoids averaging over non-max positions.

Deconvnet — Rectification

Rectification: after unpooling, apply **ReLU** to the reconstruction signal:

$$\tilde{a} \leftarrow \max(\tilde{a}, 0)$$

This ensures that only positive activations are back-projected (consistent with the forward pass convention).

Deconvnet — Filtering

Filtering (transposed convolution): apply the **transposed** (flipped) version of the original convolutional filters to reconstruct the input of the layer below:

$$\tilde{\mathbf{x}} = \mathbf{W}^\top \tilde{a}$$

where $\mathbf{W}^\top$ denotes the transposed convolution (also called "deconvolution" in the paper, though mathematically it is a cross-correlation with flipped kernels).

Deconvnet — Full Pipeline

$$\underbrace{\mathbf{x}}_{\text{input image}} \xrightarrow{\text{CNN (forward)}} \underbrace{\mathbf{a}^{(l)}}_{\text{layer } l \text{ activations}} \xrightarrow{\text{select unit}} \underbrace{\hat{\mathbf{a}}^{(l)}}_{\text{masked map}} \xrightarrow{\text{Deconvnet}} \underbrace{\mathbf{R}}_{\text{reconstruction in pixel space}}$$

Deconvnet layers (applied from layer l down to layer 1):

- Unpool → ReLU → Transposed convolution (repeat for each layer)

Result: $\mathbf{R} \in \mathbb{R}^{C \times H \times W}$ is a reconstruction in pixel space showing which input patterns caused the selected activation.

This is **not** the gradient — it is a specific heuristic reconstruction that tends to produce sharper, more interpretable visualizations.

Deconvnet — Examples

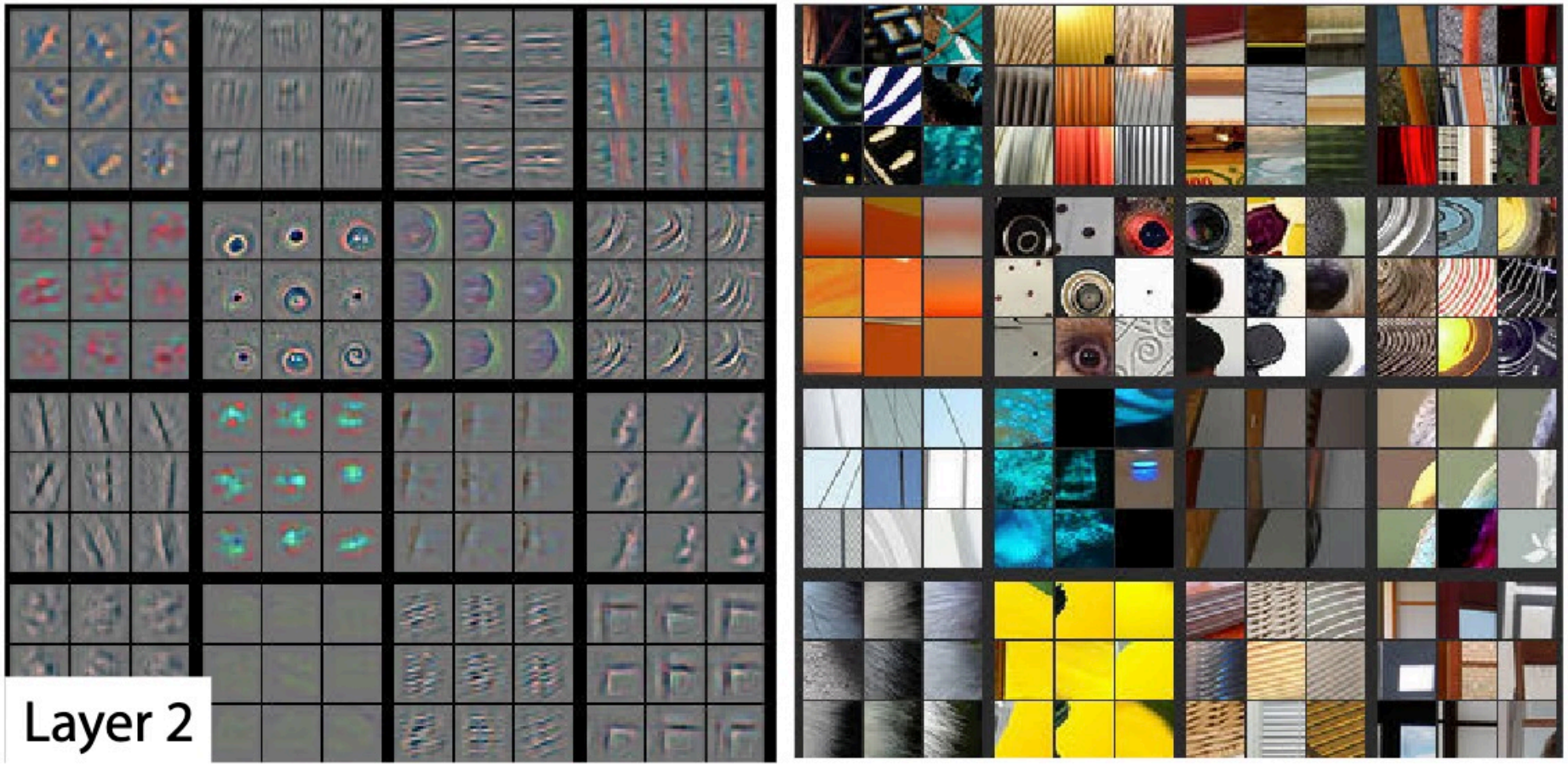


Image source: Zeiler, M. D., & Fergus, R. (2014, September). Visualizing and understanding convolutional networks. In European conference on computer vision (pp. 818-833). Cham: Springer International Publishing.

Deconvnet — Examples

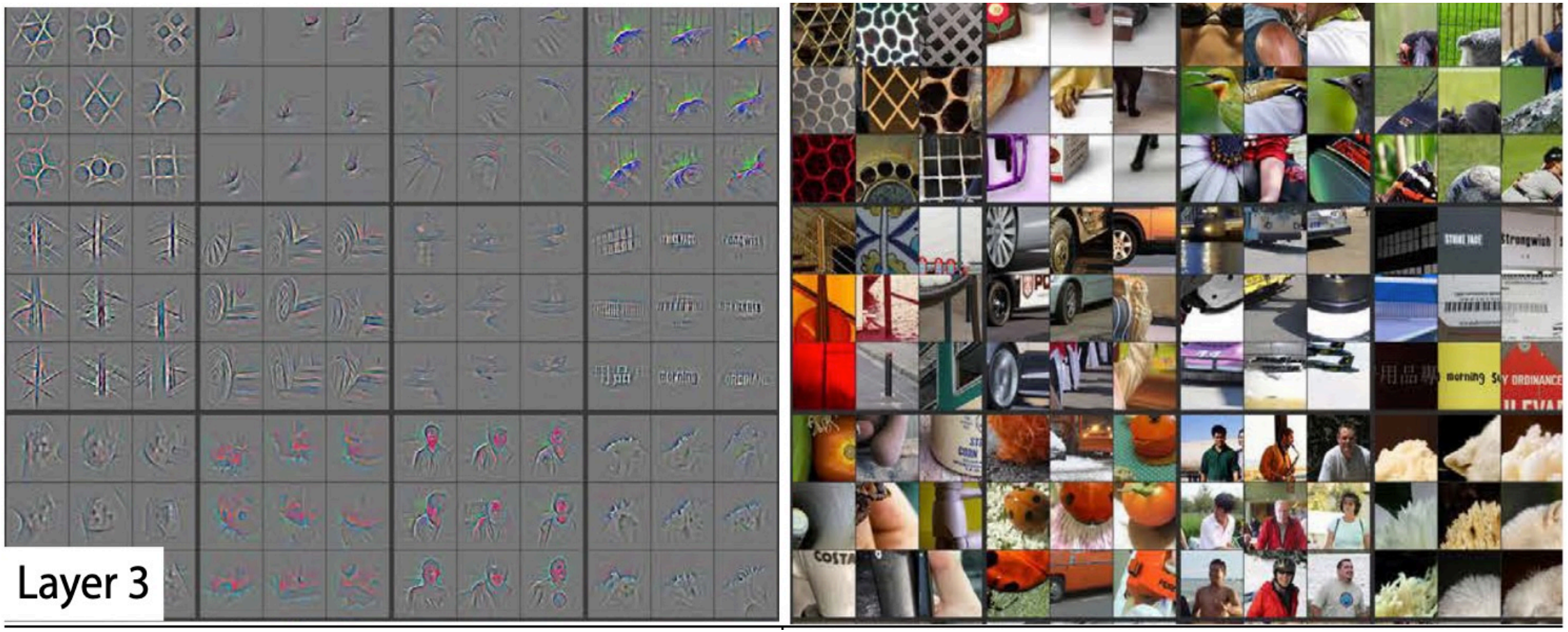


Image source: Zeiler, M. D., & Fergus, R. (2014, September). Visualizing and understanding convolutional networks. In European conference on computer vision (pp. 818-833). Cham: Springer International Publishing.

Deconvnet — Examples

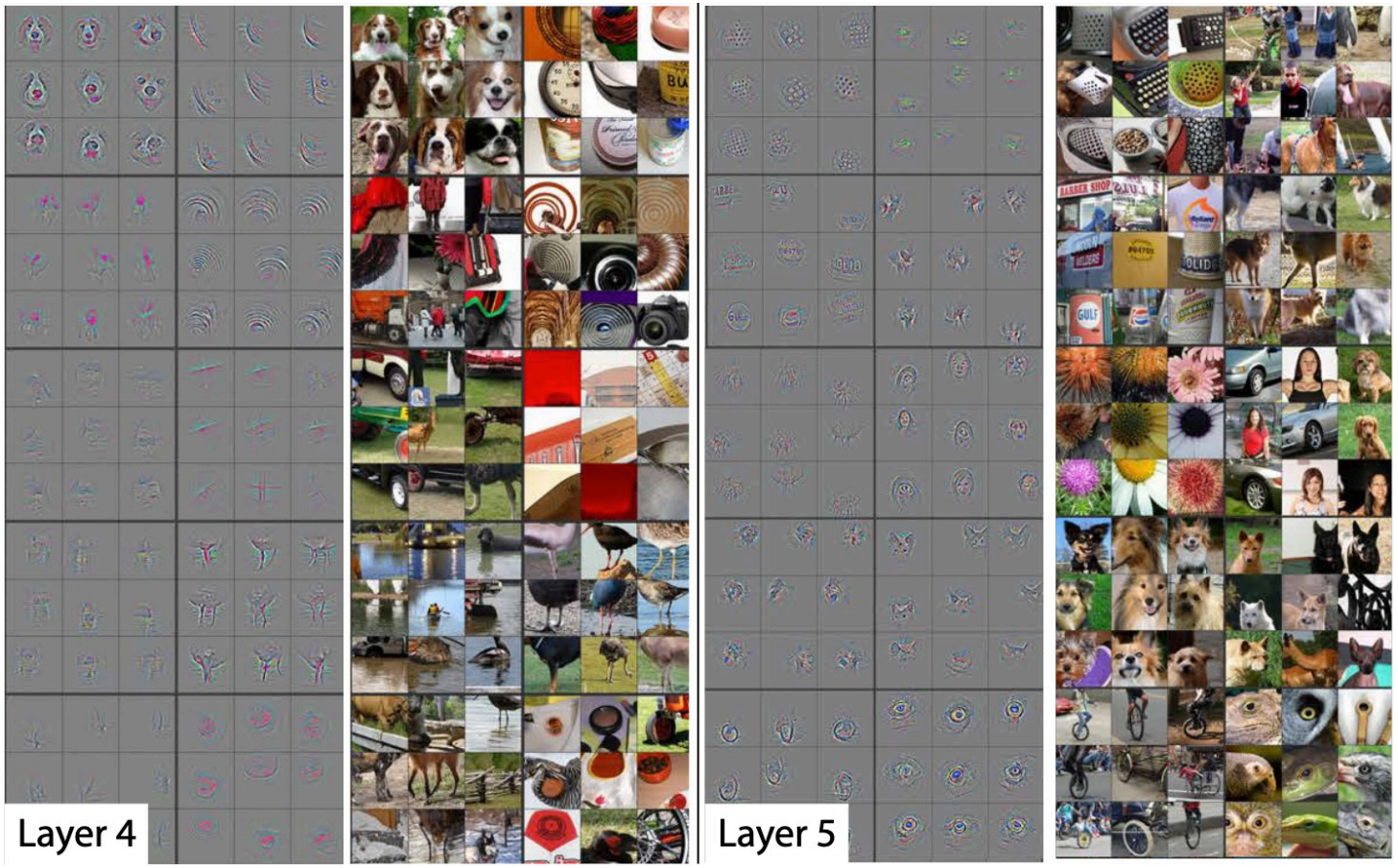


Image source: Zeiler, M. D., & Fergus, R. (2014, September). Visualizing and understanding convolutional networks. In European conference on computer vision (pp. 818-833). Cham: Springer International Publishing.

What Each Layer Learns — Zeiler & Fergus Findings

Zeiler & Fergus visualized features in AlexNet across all five convolutional layers:

Layer	Learned features
Layer 1	Oriented edges, color blobs — very similar to Gabor filters
Layer 2	Corners, junctions, simple textures (grids, stripes)
Layer 3	More complex textures, mesh patterns, text
Layer 4	Object parts (dog faces, bird legs, wheels)
Layer 5	Whole objects in various poses (keyboards, dogs, flowers)

Conclusion: the CNN automatically learns a hierarchy of visual concepts — from Gabor-like low-level features to high-level semantic representations.

Gradient-Based Visualization — Comparison Summary

Simonyan et al.:

- Single backward pass from the **output class score**
- Produces input-specific attribution maps (saliency)
- Answers: "which pixels matter for *this* prediction?"

Zeiler & Fergus:

- Backward pass from a **selected intermediate neuron**
- Requires switch variables from the forward pass
- Answers: "what input pattern made *this neuron* fire?"

CNN Visualization and the Gabor Filter Connection

Summary of the connection:

1. **Gabor filters** are optimal linear filters for representing oriented edges in natural images — studied in signal processing and believed to match receptive fields in biological vision
2. **CNNs learn Gabor-like filters** in their first layer as a result of training on natural images via gradient descent — without any explicit prior
3. **Deeper layers** learn increasingly complex features: textures, parts, objects
4. **Visualization methods** (Simonyan, Zeiler) confirm and reveal this hierarchy empirically

CNN Visualization and the Gabor Filter Connection

The fact that an engineered signal-processing approach (Gabor filters) and a learned data-driven approach (CNNs) converge to the same first-layer representation is strong evidence that Gabor-like features are a fundamental property of visual data, not an arbitrary design choice.

Practical Implications of Visualization

CNN visualization has direct practical applications:

Model debugging: Does the model filters correlate with useful features?

Dataset bias detection: Does the model learn biased filters?

Architecture search: Are any filters overfitting to specific features?

Neural network pruning: Are all neurons used or can some be removed?

Summary — What We Have Learned

Method	Paper	What it visualizes
Weight visualization	—	First-layer filter templates
Class model visualization	Simonyan et al. (2014)	Synthetic prototype per class
Gradient saliency	Simonyan et al. (2014)	Per-pixel attribution for a prediction
Deconvnet	Zeiler & Fergus (2014)	Input patterns that activate a neuron

Summary — What We Have Learned

Core message: CNNs learn a hierarchy of visual features. The first layer converges to Gabor-like oriented edge detectors — the same representation found by classical signal-processing approaches and believed to match the first stage of biological visual processing.

Visualization of CNNs and Gabor Filters

Recap!

- 🔍 **Motivation**: visualization opens the CNN black box for debugging, understanding, and trust
- 🌊 **Gabor filters**: oriented bandpass filters believed to match receptive fields in biological vision — CNNs rediscover them
- 📄 **Simonyan et al.**: gradient ascent in input space yields class prototypes; one backward pass yields saliency maps
- 🔄 **Zeiler & Fergus**: deconvnet uses unpooling + transposed convolution to reconstruct what each neuron responds to
- 🏗️ **Feature hierarchy**: edges → textures → parts → objects